

Name: Kassim Musa Abass
Reg: SPH32/3338/2025
Program: Embedded Systems Intro

Week 5: Introduction to Microcontrollers

Objective:

To understand ARM architecture fundamentals, configure GPIO pins using bare-metal register manipulation in C, and successfully run a program on actual hardware.

Introduction:

Unlike running C code on a PC, microcontrollers execute code directly on bare metal. This requires setting up a specialized toolchain, understanding the processor's memory map, and explicitly configuring peripheral registers to interact with the physical world.

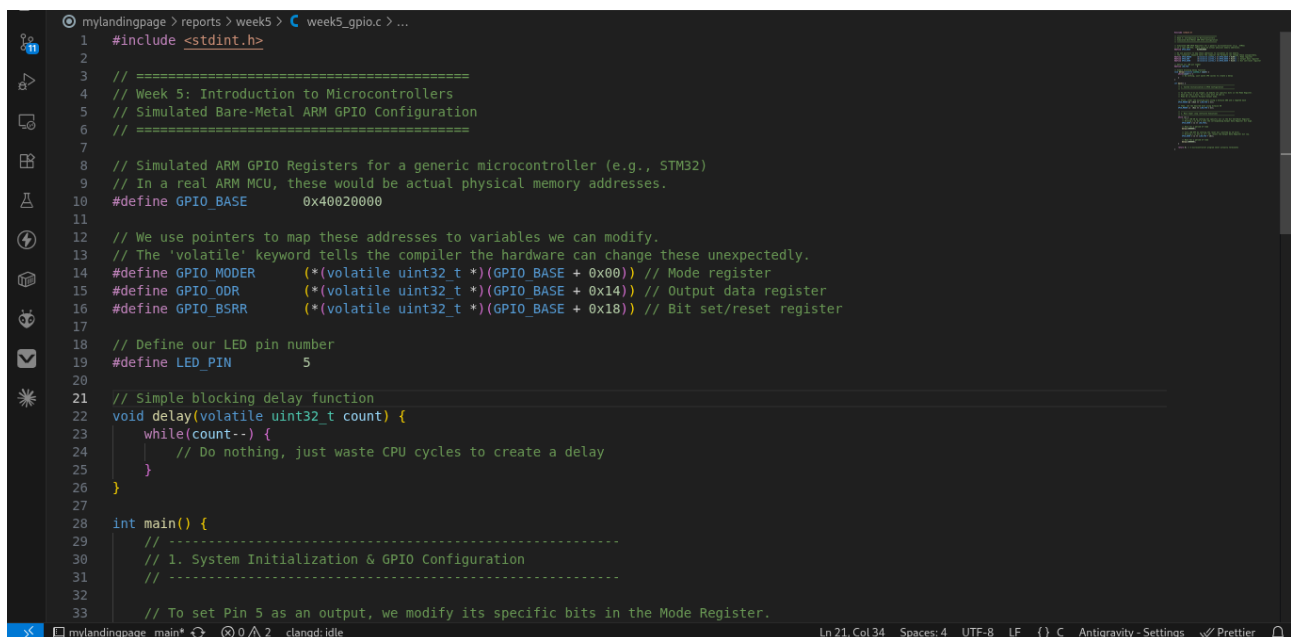
1. Theoretical Concepts

ARM Architecture Overview: ARM microcontrollers utilize a memory-mapped architecture where peripherals (like GPIO, Timers, UART) are assigned specific memory addresses.

GPIO Configuration: To control a pin, we must first configure its mode (Input/Output/Alternate Function) using the Mode Register (MODER). We achieve this by reading the register, applying a bitwise mask to clear the relevant bits, and using an OR operation to set the new mode.

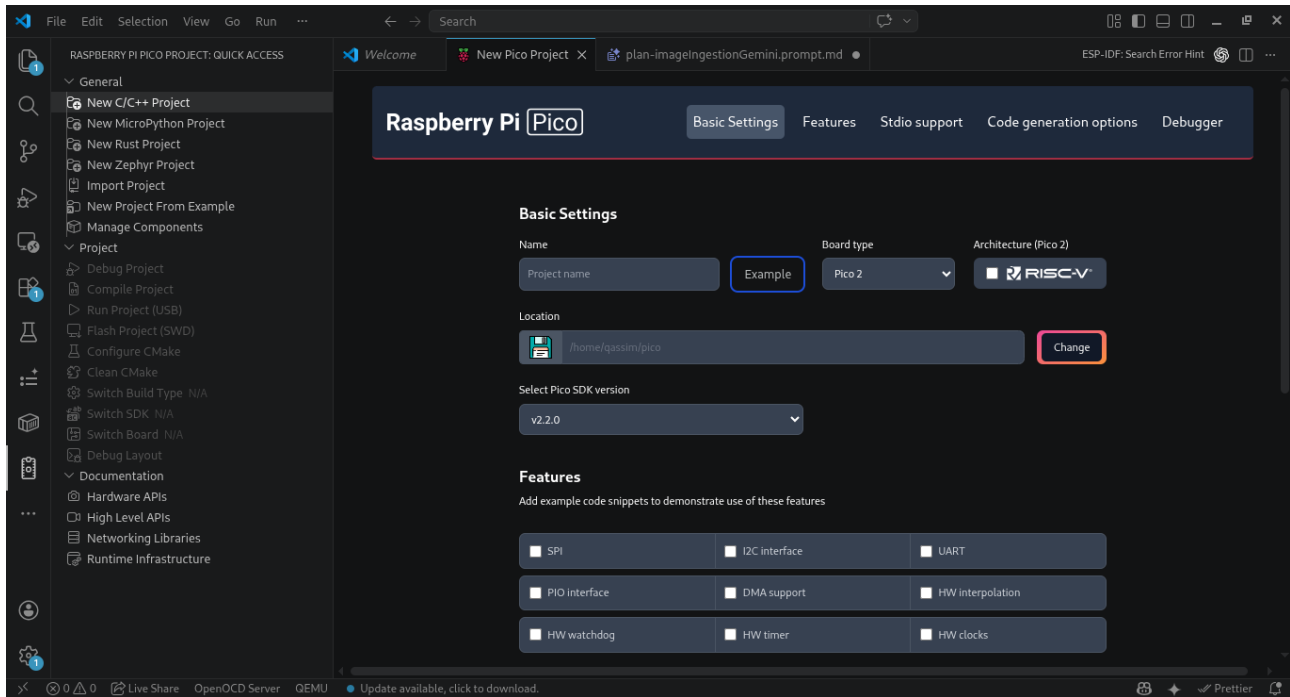
Bit Set/Reset: Rather than writing directly to the Output Data Register (ODR) which requires a read-modify-write cycle, modern ARM chips provide a Bit Set/Reset Register (BSRR) that allows atomic, thread-safe pin toggling in a single clock cycle.

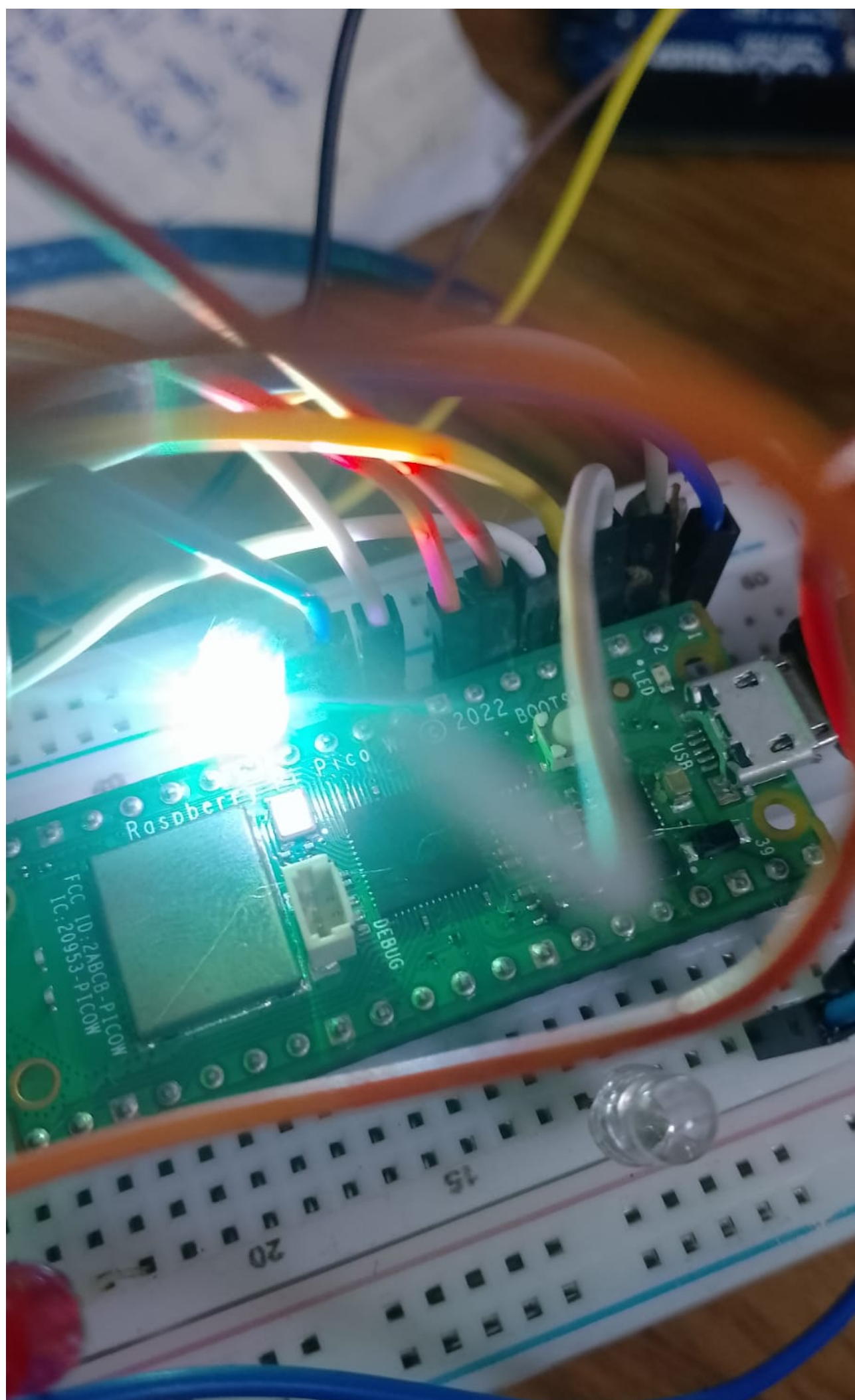
2. Source Code



```
1 #include <stdint.h>
2
3 // =====
4 // Week 5: Introduction to Microcontrollers
5 // Simulated Bare-Metal ARM GPIO Configuration
6 // =====
7
8 // Simulated ARM GPIO Registers for a generic microcontroller (e.g., STM32)
9 // In a real ARM MCU, these would be actual physical memory addresses.
10 #define GPIO_BASE 0x40020000
11
12 // We use pointers to map these addresses to variables we can modify.
13 // The 'volatile' keyword tells the compiler the hardware can change these unexpectedly.
14 #define GPIO_MODER (*(volatile uint32_t *) (GPIO_BASE + 0x00)) // Mode register
15 #define GPIO_ODR (*(volatile uint32_t *) (GPIO_BASE + 0x14)) // Output data register
16 #define GPIO_BSRR (*(volatile uint32_t *) (GPIO_BASE + 0x18)) // Bit set/reset register
17
18 // Define our LED pin number
19 #define LED_PIN 5
20
21 // Simple blocking delay function
22 void delay(volatile uint32_t count) {
23     while(count--) {
24         // Do nothing, just waste CPU cycles to create a delay
25     }
26 }
27
28 int main() {
29     // -----
30     // 1. System Initialization & GPIO Configuration
31     // -----
32
33     // To set Pin 5 as an output, we modify its specific bits in the Mode Register.
```

3. Environment & Hardware Setup





4. Reflection

Setting up the development environment required managing cross-compilers (like arm-none-eabi-gcc) and flashing tools. Writing bare-metal code is significantly different from standard PC programming because it lacks an operating system to manage hardware resources, demanding absolute precision with bitwise operations and memory addresses.